PBR VISVODAYA INSTITUTE OF TECHNOLOGY & SCIENCE

DATA STRUCTURES & ALGORITHMS

Course Code: **23CS102** • Semester: **1-2** • Department of Computer Science & Engineering

Course Scope & Objectives: Fundamental data organization structures, time/space asymptotic analysis, linear and non-linear data structures, searching, sorting, and hashing techniques.

Unit 1: Asymptotic Analysis & Linear Lists

Key Syllabus Topics: Big-O, Omega, Theta Notations, Space Complexity, Arrays as ADT, Singly Linked List, Doubly Linked List, Circular Linked List, Polynomial Representation & Addition.

Theoretical Framework & Concepts:

Asymptotic notations bound algorithmic efficiency as input size grows to infinity. Linked lists provide dynamic resizing and constant time O(1) insertions/deletions at known node positions, overcoming fixed array contiguous memory limitations.

Implementation / Key Formula Reference:

```
struct Node {
    int data;
    struct Node *next;
};
// Insertion at head: O(1) time
```

Frequently Asked University Exam Questions:

- **Q1: Compare Singly Linked List vs Doubly Linked List.**

Solution: A: SLL uses one pointer (next) per node saving memory; DLL uses two pointers (prev & next) enabling bidirectional traversal and O(1) node deletion given its pointer.

Unit 2: Stacks and Queues ADTs

Key Syllabus Topics: Stack Operations (Push, Pop, Peek), Infix to Postfix Conversion, Postfix Evaluation, Queue ADT, Circular Queue, Deque (Double Ended Queue), Priority Queue.

Theoretical Framework & Concepts:

Stacks operate on LIFO (Last-In-First-Out); used for function call stacks and syntax parsing. Standard Linear Queues suffer from false overflow due to index shifting; Circular Queues resolve this using modulo arithmetic (rear + 1) % capacity.

Implementation / Key Formula Reference:

```
int isFull() { return (rear + 1) % CAPACITY == front; }
int isEmpty() { return front == -1; }
```

Frequently Asked University Exam Questions:

- **Q1: Explain how an arithmetic infix expression is converted to postfix.**

  Solution: A: Operands are appended directly to output; operators are pushed to stack based on precedence and associativity. Parentheses enforce evaluation grouping.

Unit 3: Trees, Binary Search Trees & Balanced Trees

Key Syllabus Topics: Binary Tree Terminology, Tree Traversals (Inorder, Preorder, Postorder, Level-order), Binary Search Tree (BST) Operations, AVL Trees (LL, RR, LR, RL Rotations), B-Trees, Threaded Binary Trees.

Theoretical Framework & Concepts:

Binary Search Trees enforce left < root < right ordering. Skewed BSTs degrade to O(N) operations. AVL Trees maintain balance factor in {-1, 0, 1} through rotations, guaranteeing O(log N) worst-case search, insertion, and deletion time.

Implementation / Key Formula Reference:

```
// Inorder traversal yields sorted output for BST
void inorder(Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->val);
        inorder(root->right);
    }
}
```

Frequently Asked University Exam Questions:

- **Q1: Explain AVL Tree rotations with diagrams.**

  Solution: A: Single rotations (LL, RR) fix simple imbalance; Double rotations (LR = Left on left-child then Right on root, RL = Right on right-child then Left on root) restore height balance.

Unit 4: Graphs and Traversal Algorithms

Key Syllabus Topics: Graph Representation (Adjacency Matrix, Adjacency List), Breadth First Search (BFS), Depth First Search (DFS), Topological Sorting, Spanning Trees (Prim's & Kruskal's Algorithms), Shortest Paths (Dijkstra's).

Theoretical Framework & Concepts:

Graphs represent non-linear relationships. BFS uses a Queue for level-wise shortest paths on unweighted graphs; DFS uses a Stack/Recursion. Prim's algorithm grows a tree node-by-node, while Kruskal's selects lowest weight edges using Disjoint Set Union (DSU).

Implementation / Key Formula Reference:

```
// Dijkstra shortest path: PriorityQueue based greedy relaxation
if (dist[u] + weight < dist[v]) {
    dist[v] = dist[u] + weight;
}
```

Frequently Asked University Exam Questions:

• Q1: Differentiate between BFS and DFS with their time complexities.

Solution: A: Both run in O(V + E) with adjacency lists. BFS explores neighbors level-by-level using a queue; DFS explores branches as deep as possible before backtracking using a stack.

Unit 5: Searching, Sorting & Hashing Techniques

Key Syllabus Topics: Binary Search, Quick Sort (Partitioning), Merge Sort (Divide & Conquer), Heap Sort, Hash Tables, Hash Functions (Division, Folding), Collision Resolution (Chaining, Linear Probing, Quadratic Probing, Double Hashing).

Theoretical Framework & Concepts:

Merge sort guarantees O(N log N) worst case with O(N) auxiliary space. Quick sort achieves O(N log N) average case with in-place partitioning. Hash tables achieve average O(1) search/insert by mapping keys to array indices; collisions are handled by chaining or open addressing.

Implementation / Key Formula Reference:

```
int hash(int key) { return key % TABLE_SIZE; }
// Linear Probing: index = (hash(key) + i) % TABLE_SIZE
```

Frequently Asked University Exam Questions:

• Q1: Explain collision resolution using Separate Chaining and Open Addressing.

Solution: A: Chaining stores colliding entries in a linked list at that bucket. Open addressing searches for alternate vacant slots within the primary hash table array.